Infosys Springboard Virtual Internship 6.0 Completion Report

Team Details

Batch Number: 2
Start Date: 08
Team Name: Team 2
Team Members:
1. Vishnupriya V
2. Anureema Sen
3. Sai Keerthi Gade
4. Gayatri Chebolu
5. Leela Venkata Sai Gabbula
Internship Duration: 8 Weeks

1. Project Title

CodeGenie AI Explainer

2. Project Objective

The main objective of the CodeGenie AI Explainer project was to develop an intelligent, user-friendly platform that automatically analyzes, explains, and debugs source code using Artificial Intelligence.
The project aimed to integrate Streamlit as the frontend UI, Ollama AI models at the backend for code explanation and debugging, and OCR (Optical Character Recognition) for extracting code from documents or images.

The goal was to help users understand and fix code more effectively by providing clear AI-generated explanations directly from uploaded code or scanned sources. This solution not only enhances developer productivity but also supports students

in learning how code works.

By completing this project, the team demonstrated the integration of multiple AI technologies to create a smart and interactive code analysis tool. The project aligns with Infosys Springboard's vision of promoting AI-driven innovation and practical learning in software development.

3. Project Description

The CodeGenie AI Explainer project focuses on building an AI-powered application that simplifies the process of understanding and debugging programming code. The system provides an interactive Streamlit-based interface where users can upload code files or images containing code snippets.

At the backend, Ollama AI models process the input and generate meaningful explanations, helping users identify errors and understand code flow. The integration of OCR (Optical Character Recognition) further enhances the system by allowing users to extract code from scanned documents or handwritten notes.

The backend logic efficiently connects the frontend UI and the AI model to handle data transfer, response generation, and dynamic result display. This seamless integration provides real-time feedback, code correction support, and educational insights.

Overall, the project demonstrates the use of Machine Learning, AI models, and UI integration to create a real-world, intelligent code analysis tool. It promotes hands-on learning, teamwork, and practical application of theoretical concepts in machine learning and AI.

. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Learn about Machine Learning (ML) and its types	Gained understanding of Machine Learning algorithms — Linear Regression, Logistic Regression, Clustering, CART, SVM, Neural Networks, and Reinforcement Learning. Finalized project title "CodeGenie AI Explainer" and performed background research.
Week 2	Research Streamlit and explore its features	Studied Streamlit framework and its role in frontend development. Created documentation and delivered a presentation explaining Streamlit concepts and usage.
Week 3	Begin development of the frontend	Designed and implemented a ChatGPT-like Streamlit interface. Added input fields, buttons, and layout to support user interaction and tested UI flow.
Week 4	Understand and integrate Ollama AI model	Explored Ollama AI model structure and integrated it into the Streamlit interface for automatic code explanations. Documented the process and presented progress.
Week 5	Study and integrate OCR (Optical Character Recognition)	Learned OCR technology and implemented it to extract code from images/documents. Successfully connected

		OCR output with the Streamlit + Ollama system.
Week 6	Complete full project integration	Combined all modules — Streamlit UI, Ollama AI, and OCR — to complete the CodeGenie AI Explainer project. Finalized code, documentation, and created a team presentation.
Week 7	Apply MIT License and host project	Understood MIT License guidelines, uploaded project files to GitHub, and prepared the final CodeGenie AI Explainer demo and presentation.
Week 8	Final submission and review	Completed final report, documentation, and presentations. Submitted the complete CodeGenie AI Explainer project with all supporting files and deliverables.

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	The team discussed project ideas, finalized the title 'CodeGenie AI Explainer,' and assigned roles and responsibilities.	Week 1
Prototype / First Draft	Developed the initial Streamlit-based frontend and connected it with the Ollama AI model for code explanation.	Week 3
Mid-Term Review	Integrated OCR with Streamlit and Ollama; conducted internal testing and presented progress for feedback.	Week 5

Final Submission	Completed full project integration, documentation, and conducted accuracy comparison of Ollama with Gemini and Groq models.	Week 7
Presentation	Delivered the final team presentation and submitted the complete CodeGenie AI Explainer project with all supporting files.	Week 8

5b. Project Execution Details

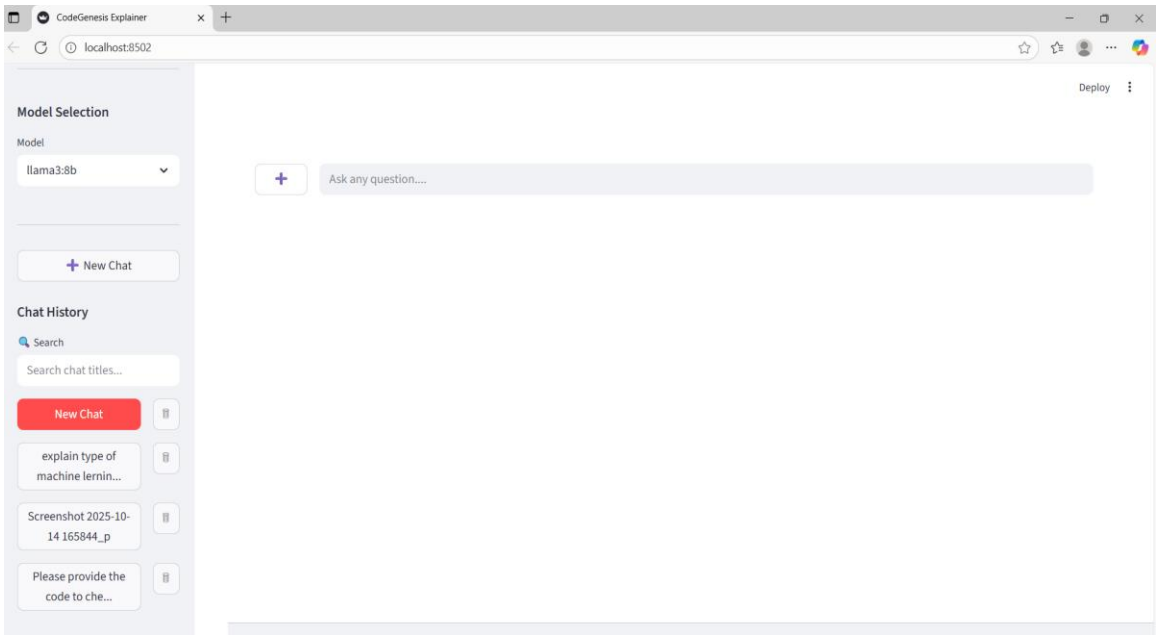
The project was executed in multiple phases, starting from concept understanding to implementation. Initially, the team learned core Machine Learning algorithms such as regression, clustering, and neural networks. After gaining the theoretical base, the focus shifted to the Streamlit frontend, where the team developed an interactive UI.

In the next phase, the Ollama AI model was integrated to process and explain code dynamically. Later, the OCR module was added, allowing text extraction from uploaded documents or handwritten notes. The entire system was tested for accuracy and performance, ensuring smooth frontend-backend interaction.

The project was documented weekly, with presentations and reviews conducted to ensure progress. The final version of CodeGenie AI Explainer was successfully completed, integrated, and demonstrated to showcase AI's real-world applications in software learning and development.

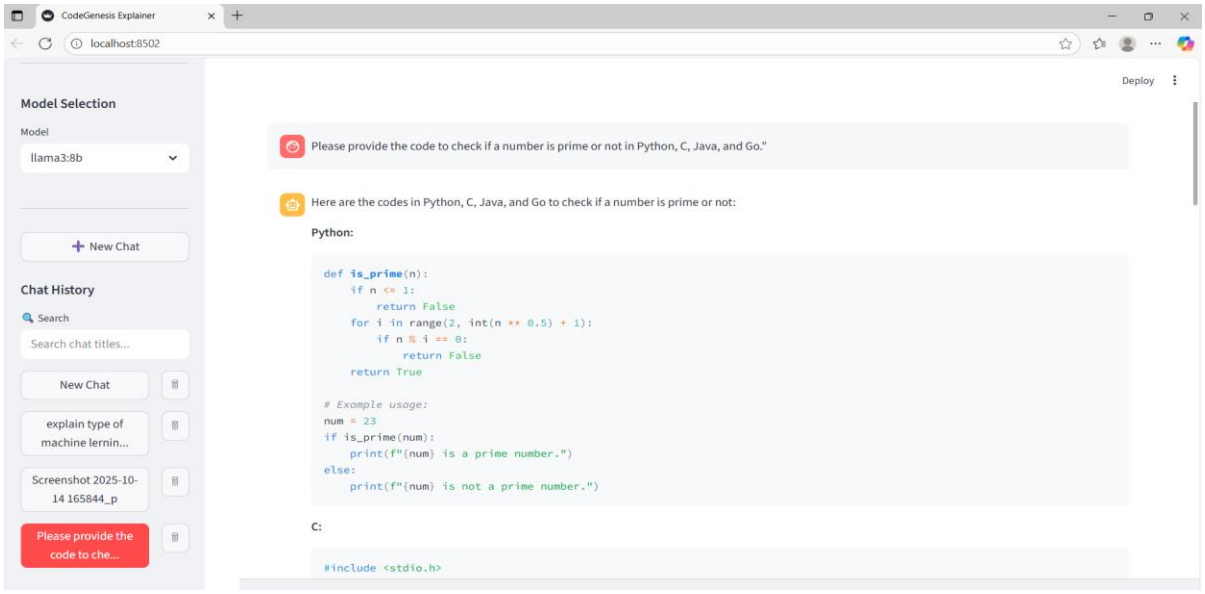
6. Snapshots / Screenshots

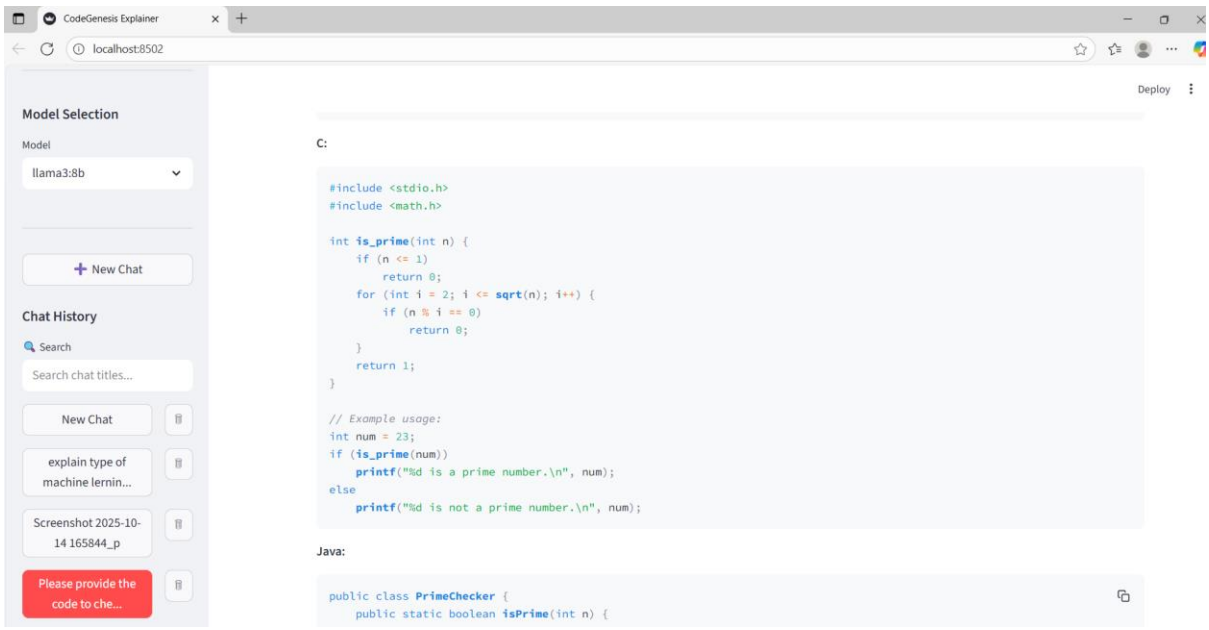
Screenshot 1: Streamlit Interface (Frontend UI)

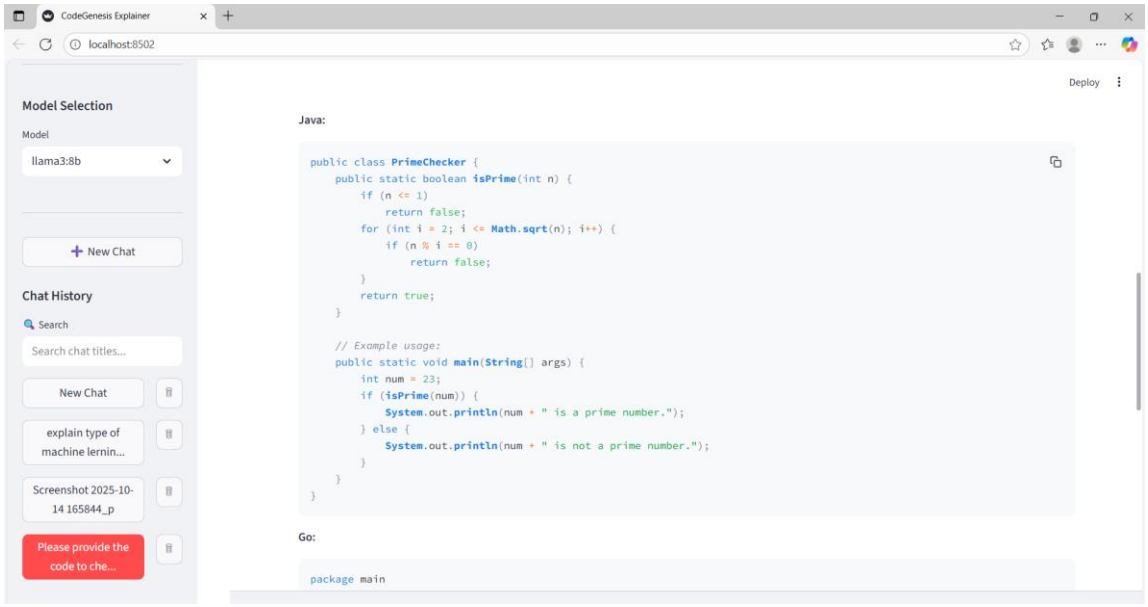


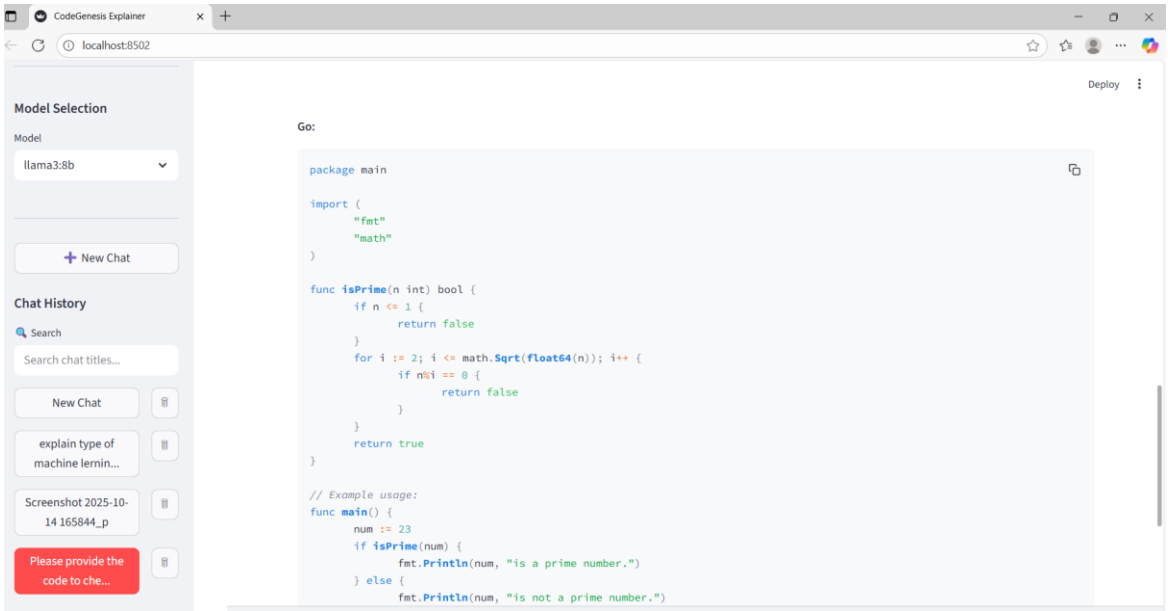
This screenshot shows the main user interface of the CodeGenie AI Explainer built using Streamlit.
The sidebar provides options for model selection and chat history, while the main panel allows users to enter code or upload files for explanation.
The “+” icon activates the OCR feature for code extraction from images or documents.
The search bar works dynamically and is case-insensitive, ensuring quick access to chats.
The “New Chat” button opens

Screenshot 2: AI Explanation Output (Ollama Integration)



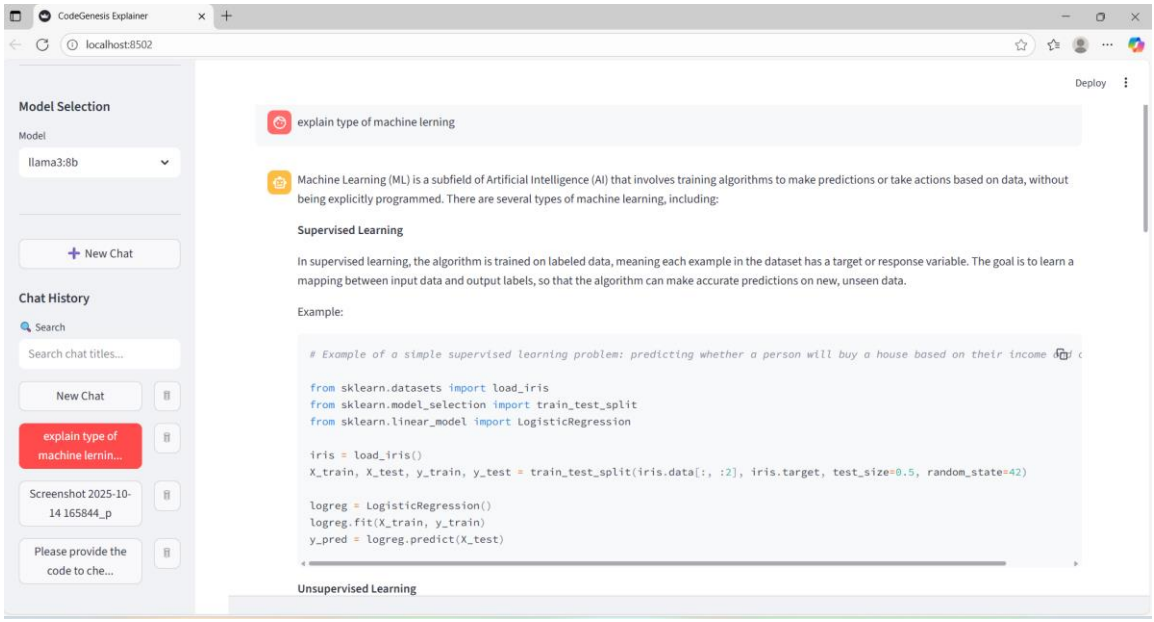



CodeGenesis Explainer
localhost:8502
Model Selection
Model
llama3:8b
New Chat
Chat History
Search
Search chat titles...
New Chat
explain type of machine lernin...
Screenshot 2025-10-14 165844_p
Please provide the code to che...
Deploy
Java:
public class PrimeChecker {
public static boolean isPrime(int n) {
if (n <= 1)
return false;
for (int i = 2; i <= Math.sqrt(n); i++) {
if (n % i == 0)
return false;
}
return true;
}
// Example usage:
public static void main(String[] args) {
int num = 23;
if (isPrime(num)) {
System.out.println(num + " is a prime number.");
} else {
System.out.println(num + " is not a prime number.");
}
}
}
Go:
package main


CodeGenesis Explainer
localhost:8502
Model Selection
Model
llama3:8b
New Chat
Chat History
Search
Search chat titles...
New Chat
explain type of machine lernin...
Screenshot 2025-10-14 165844_p
Please provide the code to che...
Deploy
Go:
package main
import (
"fmt"
"math"
)
func isPrime(n int) bool {
if n <= 1 {
return false
}
for i := 2; i <= math.Sqrt(float64(n)); i++ {
if n%i == 0 {
return false
}
}
return true
}
// Example usage:
func main() {
num := 23
if isPrime(num) {
fmt.Println(num, "is a prime number.")
} else {
fmt.Println(num, "is not a prime number.")

This screenshot illustrates how the **Ollama AI model** processes user input and provides rich, step-by-step code explanations.
When a user submits a query, the system interprets the prompt and responds with clear, structured answers across different programming languages such as **Python**, **C**, **Java**, and **Go**.
The AI not only displays the required code but also explains its logic, syntax, and purpose, helping users understand programming concepts better.
This integration demonstrates the project's ability to combine a conversational interface with real-time AI reasoning and code generation.

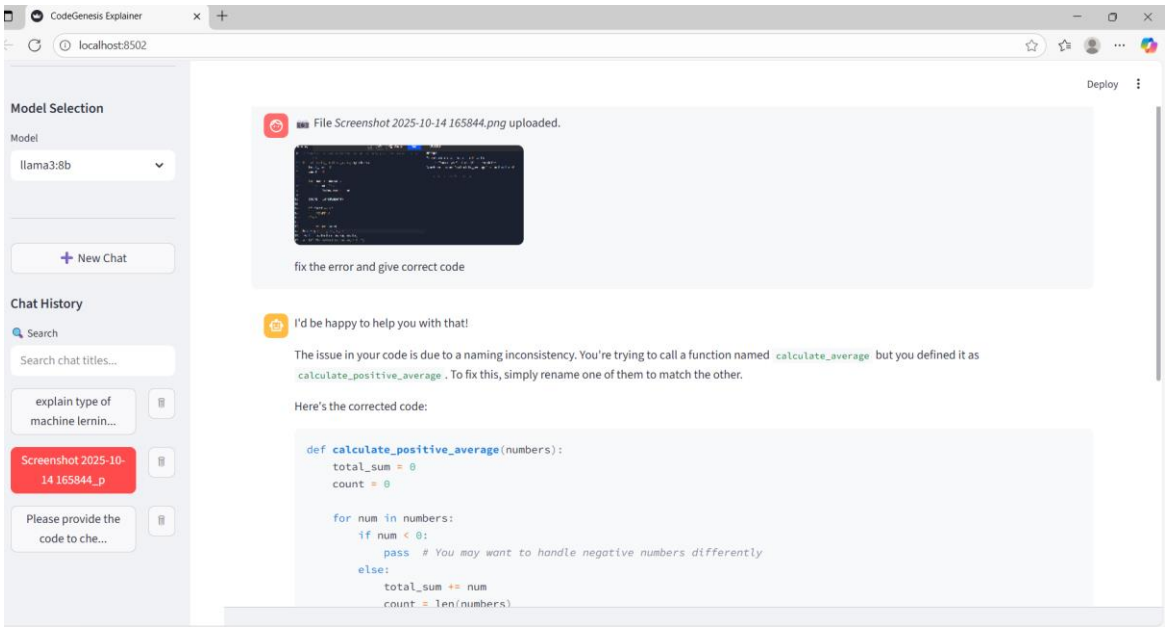
Screenshot 3: Machine Learning Concept Explanation



This example demonstrates how the AI Explainer educates users by breaking down key Machine Learning (ML) topics such as *Supervised Learning*, *Regression*, and *Classification*.
The output contains concise definitions followed by relevant Python examples, showing how these algorithms are implemented in practice.
This feature transforms CodeGenie AI Explainer into not just a debugging tool but also an AI-assisted learning companion that supports developers and students in understanding ML concepts.

It showcases the educational capability of the project alongside its technical functionality.

Screenshot 4: OCR Code Extraction and Correction



This screenshot highlights the **OCR integration** in the CodeGenie AI Explainer application.
An uploaded image containing code is automatically processed through OCR to extract the text.
Once extracted, the text is analyzed by the AI, which detects syntax or logical errors and suggests the **corrected version** of the code.
The display also includes an explanation of what changes were made and why, providing users both a fix and a learning opportunity.
This seamless link between OCR and AI shows how handwritten or scanned code snippets can be turned into executable and understandable programs.

7. Challenges Faced

- Integrating multiple technologies (Streamlit, Ollama, OCR) required learning and testing compatibility.
- Understanding how to process text extraction from OCR and feed it to AI models correctly.
- Managing version control and maintaining consistency in team coordination during code updates.
- Time management for completing documentation and presentations along with development work.

All challenges were overcome through teamwork, regular communication, and guidance from the mentor.

8. Learnings & Skills Acquired

Technical Skills:
- Machine Learning concepts (Linear Regression, Logistic Regression, Clustering, CART, SVM, Neural Networks)
- Streamlit frontend development
- Ollama AI integration and backend logic
- OCR implementation and code extraction
- GitHub version control and project licensing

Soft Skills:
- Team coordination and communication
- Time management and punctuality
- Consistency and dedication
- Presentation and documentation skills

9. Testimonials from Team

The CodeGenie AI Explainer internship was an incredible learning journey. We gained both technical and soft skills through practical implementation. Working as a team helped us understand real-world project collaboration and how AI can be used effectively in software development.

Acknowledgements

We, the team members of CodeGenie AI Explainer, would like to express our heartfelt gratitude to Infos

We sincerely thank our mentor Rojar Sir for his constant guidance, valuable insights, and encourageme

We would also like to thank our Team 2 members and team leader for their cooperation, teamwork, and

Before this internship, we had only basic knowledge of ML algorithms. But through Infosys Springboard

This internship has been a truly inspiring and enriching journey that has enhanced both our technical ex